2026-10-18 12:01:03 You

Create a detailed explanation but only point to point no extra things i want simple and clear understanding and basic practical examples for each

Level 4 — Configuration & Application Deployment

39. **Package management**
40. **Service management**
41. **User management**
42. **File & directory management**
43. **File permissions / ownership**
44. **Copy module**
45. **Template module**
46. **Lineinfile**
47. **Blockinfile**
48. **Command vs Shell**
49. **Git module**
50. **Archive / Unarchive**
51. **Cron jobs**

ChatGPT gpt-6-mini

Ansible Level 4 — Configuration & Application Deployment

Goal: Learn how to install software, configure Linux servers, deploy applications, and automate routine tasks using Ansible.

All examples assume an **Ubuntu server** and use `ansible.builtin` modules.

39. Package Management

Purpose: Install, update, or remove software packages on remote servers.

- Module: `apt` for Ubuntu/Debian; `dnf` for modern RHEL-based systems.
- Common tasks: Install Nginx, remove Apache, update packages.

Example — Install Nginx

```yaml
- name: Install Nginx
  hosts: web
  become: true
  tasks:
    - name: Install Nginx
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true
```

Important parameters

Parameter	Meaning
`name`	Package name
`state: present`	Install if missing
`state: absent`	Remove package
`state: latest`	Upgrade to latest available version
`update_cache: true`	Refresh package index

Production example: Install Nginx on 10 web servers with one playbook instead of manually installing it on each server.

---

40. Service Management

Purpose: Start, stop, restart, enable, or disable Linux services.

- Module: `ansible.builtin.service`
- Common services: Nginx, Docker, Jenkins, SSH.

Example — Start Nginx and enable it at boot

```yaml
```

```yaml
- name: Manage Nginx
  hosts: web
  become: true
  tasks:
    - name: Ensure Nginx is running
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
```

Important parameters

Parameter	Meaning
`state: started`	Start if stopped
`state: stopped`	Stop the service
`state: restarted`	Restart the service
`state: reloaded`	Reload service configuration
`enabled: true`	Start automatically at boot
`enabled: false`	Disable automatic startup

Production example: After deploying a new application configuration, restart the application service.

Remember: `state: started` ensures the service is running. It does not restart a service that is already running.

---

41. User Management

Purpose: Create users, remove users, manage groups, and configure login shells.

- Module: `ansible.builtin.user`
- Common use cases: Create application users, deployment users, and restricted service accounts.

Example — Create a deployment user

```yaml
</> YAML
```

```yaml
- name: Create deployment user
  hosts: web
  become: true
  tasks:
    - name: Create deploy user
      ansible.builtin.user:
        name: deploy
        shell: /bin/bash
        groups: sudo
        append: true
        state: present
```

Important parameters

Parameter	Meaning
`name`	Username
`shell`	Login shell
`groups`	Supplementary groups
`append: true`	Add groups without removing existing memberships
`create_home: true`	Create home directory
`state: present`	Ensure user exists
`state: absent`	Remove user

Production example: Create a dedicated `appuser` account to run an application instead of running it as root.

Important: Adding a user to `sudo` grants elevated privileges on Ubuntu. Grant only the permissions required.

---

42. File & Directory Management

Purpose: Create files and directories, remove them, and ensure a particular path exists.

- Module: `ansible.builtin.file`

Example — Create an application directory

```yaml
- name: Create application directory
  hosts: web
  become: true
  tasks:
    - name: Create app directory
      ansible.builtin.file:
        path: /opt/myapp
        state: directory
        owner: deploy
        group: deploy
        mode: '0755'
```

Important parameters

Parameter	Meaning
`path`	File or directory path
`state: directory`	Ensure directory exists
`state: touch`	Create an empty file if missing or update its timestamp
`state: absent`	Remove file or directory
`state: file`	Manage an existing file's attributes; does not create a missing file
`recurse: true`	Apply specified attributes recursively

Example — Create an empty file

```yaml
- name: Create application log file
  ansible.builtin.file:
    path: /var/log/myapp.log
    state: touch
```

Production example: Ensure `/opt/myapp`, `/var/log/myapp`, and `/etc/myapp` exist before deploying an application.

43. File Permissions / Ownership

Purpose: Control who can read, write, or execute files and directories.

- Module: `ansible.builtin.file`
- Main parameters: `owner`, `group`, `mode`.

Example — Secure an application configuration file

```yaml
- name: Set configuration permissions
  ansible.builtin.file:
    path: /etc/myapp/config.yml
    owner: root
    group: root
    mode: '0640'
```

Linux permission reference

Permission	Numeric value	Meaning
Read (`r`)	4	Read content
Write (`w`)	2	Modify content
Execute (`x`)	1	Execute file or access directory
`0755`	Owner: 7, group: 5, others: 5	Owner can modify; others can read/execute
`0644`	Owner: 6, group: 4, others: 4	Owner can write; others can read
`0600`	Owner: 6, group: 0, others: 0	Owner only
`0640`	Owner: 6, group: 4, others: 0	Owner can read/write; group can read

Remember: `mode` should generally be quoted, for example `'0644'`.

Production example: Use `'0600'` for a private key file so only its owner can read or modify it.

---

44. Copy Module

Purpose: Copy files from the Ansible control node to remote servers.

- Module: `ansible.builtin.copy`
- Typical use: Copy static configuration files, scripts, certificates, or application assets.

Example — Copy an Nginx configuration file

```yaml
- name: Copy Nginx configuration
  hosts: web
  become: true
  tasks:
    - name: Deploy configuration
      ansible.builtin.copy:
        src: files/nginx.conf
        dest: /etc/nginx/nginx.conf
        owner: root
        group: root
        mode: '0644'
```

Folder structure

```
ansible-project/
├──── deploy.yml
└──── files/
    └──── nginx.conf
```

Important parameters

Parameter	Meaning
`src`	Source file on the control node
`dest`	Destination path on the remote server
`owner`	File owner
`group`	File group
`mode`	File permissions

Parameter	Meaning
`backup: true`	Back up an existing destination before replacing it

Production example: Deploy a static Nginx configuration to all web servers.

Remember: Use `copy` when the file content is static. Use `template` when the content needs variables.

---

45. Template Module

Purpose: Generate configuration files dynamically using variables and Jinja2 expressions.

- Module: `ansible.builtin.template`
- File extension: `.j2`

Example — Generate an Nginx configuration

`templates/nginx.conf.j2`

```nginx
server {
    listen {{ nginx_port }};
    server_name {{ server_name }};

    location / {
        root /var/www/html;
        index index.html;
    }
}
```

Playbook:

```yaml
- name: Configure Nginx
  hosts: web
  become: true
  vars:
    nginx_port: 8080
    server_name: example.com

  tasks:
    - name: Generate Nginx configuration
```

```
    ansible.builtin.template:
      src: templates/nginx.conf.j2
      dest: /etc/nginx/conf.d/myapp.conf
      owner: root
      group: root
      mode: '0644'
```

Folder structure

```
</> Plain text

ansible-project/
├──── deploy.yml
└──── templates/
    └──── nginx.conf.j2
```

How it works

1. Ansible reads the `.j2` template.
2. It replaces `{{ nginx_port }}` with `8080`.
3. It replaces `{{ server_name }}` with `example.com`.
4. It writes the rendered configuration to the remote server.

Production example: Generate different application configurations for development, staging, and production using different variable values.

Remember: Do not put passwords or secrets in templates without an appropriate secret-management approach, such as Ansible Vault.

---

46. Lineinfile Module

Purpose: Add, replace, or remove a specific line in a file.

- Module: `ansible.builtin.lineinfile`
- Typical use: Modify one setting in an existing configuration file.

Example — Change SSH configuration

```
</> YAML

- name: Configure SSH
  hosts: web
  become: true
  tasks:
```

```
    - name: Disable SSH password authentication
      ansible.builtin.lineinfile:
        path: /etc/ssh/sshd_config
        regexp: '^#?PasswordAuthentication'
        line: 'PasswordAuthentication no'
```

Important parameters

Parameter	Meaning
`path`	File to modify
`line`	Desired line
`regexp`	Pattern used to find an existing line
`state: present`	Ensure the line exists
`state: absent`	Remove matching lines
`create: true`	Create the file if missing

How it works

- If the matching line exists, Ansible replaces it.
- If no matching line exists, Ansible normally adds the desired line.
- Running the playbook again does not keep adding duplicate matching lines.

Production example: Change a configuration setting without replacing the entire configuration file.

Caution: Validate SSH configuration and retain a working access session before applying SSH changes to remote servers.

---

47. Blockinfile Module

Purpose: Add, update, or remove a complete block of text inside a file.

- Module: `ansible.builtin.blockinfile`
- Typical use: Insert multiple configuration lines together.

Example — Add a custom Nginx configuration block

```yaml
- name: Add custom Nginx settings
  hosts: web
  become: true
  tasks:
    - name: Insert configuration block
      ansible.builtin.blockinfile:
        path: /etc/nginx/conf.d/custom.conf
        create: true
        block: |
          server {
              listen 8080;
              server_name example.com;
          }
```

Ansible manages the block using marker comments similar to:

```nginx
# BEGIN ANSIBLE MANAGED BLOCK
server {
    listen 8080;
    server_name example.com;
}
# END ANSIBLE MANAGED BLOCK
```

Important parameters

Parameter	Meaning
`path`	File to modify
`block`	Multiline text to insert
`marker`	Text identifying the managed block
`state: present`	Ensure the block exists
`state: absent`	Remove the managed block
`create: true`	Create the file if missing

Production example: Manage a group of custom application settings without overwriting unrelated configuration.

Remember:

- `lineinfile` → Manage one line.
- `blockinfile` → Manage multiple lines as one block.
- `template` → Generate an entire file dynamically.

---

48. Command vs Shell

Purpose: Execute commands on remote servers.

Modules: `ansible.builtin.command` and `ansible.builtin.shell`.

A. Command module

Runs a command directly without a shell.

```yaml
</> YAML

- name: Check disk usage
  ansible.builtin.command: df -h
  register: disk_result

- name: Display disk usage
  ansible.builtin.debug:
    var: disk_result.stdout
```

B. Shell module

Runs a command through the remote shell, allowing shell features such as pipes, redirection, and environment-variable expansion.

```yaml
</> YAML

- name: Find large log files
  ansible.builtin.shell: "du -ah /var/log | sort -h | tail -5"
  register: log_result

- name: Display largest log files
  ansible.builtin.debug:
    var: log_result.stdout
```

Comparison

Feature	`command`	`shell`
Runs normal commands	Yes	Yes
Pipes (`\	`)	No shell interpretation
Redirection (`>`)	No shell interpretation	Yes
Shell variables (`$HOME`)	No shell expansion	Yes
Security exposure	Generally lower	Greater if input is untrusted
Preferred default	Yes	Only when shell features are needed

Important: Use `ansible.builtin.command` by default. Use `ansible.builtin.shell` only when shell syntax is required.

Better approach: For installing packages, managing files, or starting services, prefer their dedicated Ansible modules over commands.

Production example: Use `command` to run a migration script, or `shell` when a pipeline is genuinely necessary.

---

49. Git Module

Purpose: Clone a repository, update its code, or check out a specific branch, tag, or commit.

- Module: `ansible.builtin.git`
- Typical use: Deploy application code from GitHub to a server.

Example — Clone an application repository

```yaml
- name: Deploy application code
  hosts: web
  become: true
  tasks:
    - name: Clone application repository
      ansible.builtin.git:
        repo: https://github.com/example/myapp.git
        dest: /opt/myapp
```

```
        version: main
        update: true
```

Important parameters

Parameter	Meaning
`repo`	Repository URL
`dest`	Local checkout directory on the remote host
`version`	Branch, tag, or commit
`update: true`	Update an existing checkout
`force: true`	Force checkout changes, potentially discarding local modifications

How it works

1. If the destination does not contain a repository, Ansible clones it.
2. If the repository already exists, Ansible can update it.
3. Ansible checks out the requested version.

Production example: Deploy a specific release tag rather than always deploying the latest code from `main`.

Remember: For private repositories, configure appropriate SSH keys or another secure authentication method. Avoid embedding access tokens directly in the repository URL.

---

50. Archive / Unarchive

Purpose: Compress files or extract an existing archive.

- `ansible.builtin.archive` → Create an archive.
- `ansible.builtin.unarchive` → Extract an archive.

A. Unarchive — Extract an application package

```
</> YAML

- name: Extract application
  hosts: web
```

```yaml
  become: true
  tasks:
    - name: Extract release archive
      ansible.builtin.unarchive:
        src: /tmp/myapp.tar.gz
        dest: /opt/myapp
        remote_src: true
```

Here, `remote_src: true` means the archive already exists on the remote server.

B. Copy an archive from the control node and extract it

```yaml
- name: Extract local release archive
  ansible.builtin.unarchive:
    src: files/myapp.tar.gz
    dest: /opt/myapp
    remote_src: false
```

With `remote_src: false`, the archive is transferred from the control node before extraction.

C. Archive — Create a compressed archive

```yaml
- name: Compress application logs
  ansible.builtin.archive:
    path: /var/log/myapp
    dest: /tmp/myapp-logs.tar.gz
    format: gz
```

Important parameters

Parameter	Meaning
`src`	Archive source for `unarchive`
`dest`	Extraction directory or archive output path
`remote_src: true`	Source archive is on the remote host
`remote_src: false`	Source archive is on the control node
`path`	Files or directories to archive

Parameter	Meaning
`format: gz`	Create a gzip-compressed archive

Production example: Transfer a packaged application release, extract it into a release directory, and configure the service to use that release.

---

51. Cron Jobs

Purpose: Schedule commands or scripts to run automatically at specific times.

- Module: `ansible.builtin.cron`
- Typical use: Backups, log cleanup, periodic health checks.

Example — Run a backup script every day at 2 AM

```yaml
- name: Schedule backup
  hosts: web
  become: true
  tasks:
    - name: Create daily backup cron job
      ansible.builtin.cron:
        name: Daily application backup
        minute: "0"
        hour: "2"
        job: "/usr/local/bin/backup.sh"
        user: root
        state: present
```

Cron syntax

```
minute hour day month weekday
```

Schedule	Meaning
`0 2 * * *`	Every day at 2 AM
`*/5 * * * *`	Every 5 minutes

Schedule	Meaning
`0 9 * * 1`	Every Monday at 9 AM
`30 18 * * *`	Every day at 6:30 PM

Important parameters

Parameter	Meaning
`name`	Identifies the managed cron job
`minute`	Minute of execution
`hour`	Hour of execution
`job`	Command or script to execute
`user`	Linux user whose crontab is managed
`state: present`	Ensure the job exists
`state: absent`	Remove the managed job

Production example: Schedule nightly database backups and write output to a log file.

```yaml
job: "/usr/local/bin/backup.sh >> /var/log/backup.log 2>&1"
```

This redirects standard output and errors to the log file. The command is interpreted by the cron execution environment, which uses a shell.

Remember: Cron uses the server's configured timezone. Verify the schedule and the permissions of the script and its log directory.

Quick revision — All 13 concepts

#	Concept	Module	Main purpose
39	Package management	`apt` , `dnf`	Install/remove software
40	Service management	`service` , `systemd`	Start/stop services
41	User management	`user`	Create/manage users
42	File & directory management	`file`	Create/remove paths
43	Permissions / ownership	`file`	Manage owner, group, mode
44	Copy	`copy`	Transfer static files
45	Template	`template`	Generate variable-based files
46	Lineinfile	`lineinfile`	Manage one line
47	Blockinfile	`blockinfile`	Manage a text block
48	Command vs Shell	`command` , `shell`	Execute commands
49	Git	`git`	Clone/update repositories
50	Archive / Unarchive	`archive` , `unarchive`	Compress/extract files
51	Cron jobs	`cron`	Schedule recurring tasks

Minimal hands-on practice

Try these five tasks on one test VM. You do not need a separate lab for every module.

- [ ] Install Nginx and ensure it starts at boot.
- [ ] Create an `appuser` and `/opt/myapp` with the correct ownership and permissions.
- [ ] Copy a static file, then create a second version using a Jinja2 template.
- [ ] Update one line in a configuration file and insert a multiline block into another.
- [ ] Clone a test Git repository, extract a sample archive, and schedule a harmless cron job.

Interview priority: Understand `state` and idempotency, `copy` vs `template`, `lineinfile` vs `blockinfile`, `command` vs `shell`, and how to deploy code while ensuring the application service is running.